TEMAS SELECTOS DE CÓMPUTO DE ALTO RENDIMIENTO PARA SIMULACIONES FÍSICAS
Analisis Numérico y Computacional
Ecuacion de Calor/Laplace en 1D y 2D
Dr. Juan Carlos Cajas

Autor: Emilio Ramirez

30 de abril de 2026

Resumen

Este documento presenta la formulación matemática, discretización por diferencias finitas y evaluacion computacional de solucionadores para ecuaciones de tipo calor/Laplace. Se usa el algoritmo de Thomas para sistemas tridiagonales y se analiza el rendimiento de la paralelización con OpenMP en el caso bidimensional.

1. Introducción

El objetivo es documentar una linea base reproducible para comparar variantes de implementacion sin cambiar el paradigma matemático ni de paralelización.

2. Modelo matemático

2.1. Caso 1D estacionario

$$\frac{d}{dx}\left(k\frac{dT}{dx}\right) + \dot{q} = 0 \tag{1}$$

Para k constante (material isotrópico y ΔT pequeños) y sin fuentes internas ($\dot{q} = 0$), se obtiene la Ecuación de Laplace:

$$\frac{d^2T}{dx^2} = 0 \tag{2}$$

2.2. Caso 2D estacionario

Para el caso bidimensional estacionario, se obtiene la ecuación de Laplace en 2D:

$$\frac{\partial}{\partial x}\left(k\frac{\partial T}{\partial x}\right) + \frac{\partial}{\partial y}\left(k\frac{\partial T}{\partial y}\right) + \dot{q} = 0 \tag{3}$$

$$\Rightarrow \frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 \tag{4}$$

$$\Rightarrow \nabla^2 T = 0 \tag{5}$$

2.3. Condiciones de frontera

Se consideran condiciones de frontera de tipo Dirichlet (temperatura fija) y Neumann (flujo de calor especificado) según el experimento.

Frontera tipo Dirichlet:

$$T(0) = T_0, \quad T(L) = T_L \tag{6}$$

Frontera tipo Neumann:

Si q_N representa el flujo de calor saliente normal a la frontera, la condición de Neumann se escribe como:

$$-k\nabla T \cdot \mathbf{n} = q_N. \tag{7}$$

En el caso unidimensional, con normal exterior $\mathbf{n} = -1$ en $x = 0$ y $\mathbf{n} = 1$ en $x = L$, esto equivale a:

$$k\frac{dT}{dx}\bigg|_{x=0} = q_0, \quad -k\frac{dT}{dx}\bigg|_{x=L} = q_L. \tag{8}$$

3. Discretizacion del caso 1D

La ecuación de Laplace se discretiza usando diferencias finitas.

3.1. Ecuación interior

Se parte de la ecuación diferencial Eq. (2) discretizada:

$$\frac{\Delta}{\Delta x}\left(\frac{\Delta T}{\Delta x}\right) = 0 \tag{9}$$

$$\Rightarrow \frac{\Delta}{\Delta x}\left(\frac{T(i+1) - T(i)}{\Delta x}\right) = 0 \tag{10}$$

$$\Rightarrow \frac{\Delta T(i+1) - \Delta T(i)}{(\Delta x)^2} = 0 \tag{11}$$

$$\Rightarrow \frac{(T(i+2) - T(i+1)) - (T(i+1) - T(i))}{(\Delta x)^2} = 0 \tag{12}$$

$$\Rightarrow T_{i+2} - 2T_{i+1} + T_i = 0 \tag{13}$$

Cambiando el índice $i + 1 \to i$:

$$T_{i+1} - 2T_i + T_{i-1} = 0 \tag{14}$$

con condiciones de frontera de tipo Dirichlet/Neumann según el experimento.

Ecuación matricial 1D:

La Eq. (13) se puede escribir en forma matricial $A\mathbf{T} = \mathbf{r}$, donde A es una matriz tridiagonal con -2 en la diagonal principal y 1 en las diagonales adyacentes, $\mathbf{T}$ es el vector de temperaturas desconocidas y $\mathbf{r}$ es el vector de condiciones de frontera.

$$\begin{pmatrix} 1 & -2 & 1 & 0 & 0\cdots & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0\cdots & 0 & 0 & 0 \\ 0 & 0 & 1 & -2 & 1\cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & & & \\ 0 & 0 & 0 & 0 & 0\cdots & 1 & -2 & 1 \end{pmatrix} \begin{pmatrix} T_2 \\ T_3 \\ T_4 \\ \vdots \\ T_{n-1} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \tag{15}$$

Se hace notar que faltan las temperaturas frontera i.e. T_1 y T_n que dependerán del tipo de frontera (Dirichlet o Neumann).

Ecuación matricial 1D con condiciones frontera tipo Dirichlet:

Con condiciones de frontera de tipo Dirichlet, el sistema se modifica para incluir los valores conocidos en las matrices. Dado que T_1 y T_n son conocidos:

$$T_1 = T_0, \quad T_n = T_L \tag{16}$$

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0\cdots & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0\cdots & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0\cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & & & \\ 0 & 0 & 0 & 0 & 0\cdots & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 0\cdots & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} T_1 \\ T_2 \\ T_3 \\ \vdots \\ T_{n-1} \\ T_n \end{pmatrix} = \begin{pmatrix} T_0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ T_L \end{pmatrix} \tag{17}$$

Ecuación matricial 1D con frontera izquierda tipo Neumann:

Se estudia el caso en el que la frontera izquierda usa una condición de Neumann y la frontera derecha conserva una condición de Dirichlet. Con flujo saliente normal q_0 en $x = 0$:

$$k \frac{dT}{dx}\bigg|_{x=0} = q_0. \tag{18}$$

Aproximando la derivada normal con una diferencia hacia adelante:

$$\frac{dT}{dx}\bigg|_{x=0} \approx \frac{T_2 - T_1}{\Delta x}, \tag{19}$$

se obtiene:

$$k \frac{T_2 - T_1}{\Delta x} = q_0 \quad \Rightarrow \quad -T_1 + T_2 = \frac{q_0}{k} \Delta x. \tag{20}$$

Por tanto, el sistema algebraico para una frontera izquierda Neumann y una frontera derecha Dirichlet $T_n = T_L$ queda:

$$\begin{pmatrix} -1 & 1 & 0 & 0 & 0\cdots & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0\cdots & 0 & 0 & 0 \\ 0 & 1 & -2 & 1 & 0\cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & & & \\ 0 & 0 & 0 & 0 & 0\cdots & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 0\cdots & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} T_1 \\ T_2 \\ T_3 \\ \vdots \\ T_{n-1} \\ T_n \end{pmatrix} = \begin{pmatrix} \frac{q_0}{k}\Delta x \\ 0 \\ 0 \\ \vdots \\ 0 \\ T_L \end{pmatrix}. \tag{21}$$

Si se imponen condiciones de Neumann en ambos extremos para la ecuación de Laplace sin fuentes, el sistema queda determinado solo hasta una constante aditiva y requiere una condición adicional de referencia. Además, los flujos deben cumplir la condición de compatibilidad $q_0 + q_L = 0$.

4. Algoritmo de Thomas

Tenemos el sistema tridiagonal $A\mathbf{T} = \mathbf{r}$, donde A es una matriz tridiagonal con elementos a_i en la subdiagonal, b_i en la diagonal, y c_i en la superdiagonal. De la misma manera, $\mathbf{T}$ es el vector de temperaturas desconocidas y $\mathbf{r}$ es el vector de condiciones de frontera, con elementos d_i.

$$\begin{aligned} b_1 T_1 + c_1 T_2 &= r_1 \\ a_2 T_1 + b_2 T_2 + c_2 T_3 &= r_2 \\ a_3 T_2 + b_3 T_3 + c_3 T_4 &= r_3 \\ &\vdots \\ a_{n-1} T_{n-2} + b_{n-1} T_{n-1} + c_{n-1} T_n &= r_{n-1} \\ a_n T_{n-1} + b_n T_n &= r_n \end{aligned}$$

$$\begin{pmatrix} b_1 & c_1 & 0 & 0 & 0\cdots & 0 & 0 & 0 \\ a_2 & b_2 & c_2 & 0 & 0\cdots & 0 & 0 & 0 \\ 0 & a_3 & b_3 & c_3 & 0\cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & & & \\ 0 & 0 & 0 & 0 & 0\cdots & a_{n-1} & b_{n-1} & c_{n-1} \\ 0 & 0 & 0 & 0 & 0\cdots & 0 & a_n & b_n \end{pmatrix} \begin{pmatrix} T_1 \\ T_2 \\ T_3 \\ \vdots \\ T_{n-1} \\ T_n \end{pmatrix} = \begin{pmatrix} r_1 \\ r_2 \\ r_3 \\ \vdots \\ r_{n-1} \\ r_n \end{pmatrix} \tag{22}$$

Así, se tienen las ecuaciones del sistema tridiagonal a resolver:

$$a_i T_{i-1} + b_i T_i + c_i T_{i+1} = r_i, \quad i = 1, 2, \ldots, n, \quad a_1 = 0, \quad c_n = 0. \tag{23}$$

Resolviendo,

$$T_1 = \frac{r_1 - c_1 T_2}{b_1} \tag{24}$$

Y definimos,

$$\tilde{r_1} = \frac{r_1}{b_1}, \quad \tilde{c_1} = \frac{c_1}{b_1} \tag{25}$$

Con lo cual, la primera ecuación se reescribe como:

$$T_1 = \tilde{r_1} - \tilde{c_1} T_2 \tag{26}$$

Si ahora resolvemos el segundo renglón para T_2:

$$T_2 = \frac{r_2 - a_2 T_1 - c_2 T_3}{b_2} \tag{27}$$

$$\Rightarrow T_2 = \frac{r_2 - a_2(\tilde{r_1} - \tilde{c_1} T_2) - c_2 T_3}{b_2} \tag{28}$$

$$\Rightarrow T_2 \left(1 - \frac{a_2 \tilde{c_1}}{b_2}\right) = \frac{r_2 - a_2 \tilde{r_1} - c_2 T_3}{b_2} \tag{29}$$

$$\Rightarrow T_2 = \frac{r_2 - a_2 \tilde{r_1} - c_2 T_3}{b_2 - a_2 \tilde{c_1}} \tag{30}$$

Y si ahora definimos

$$\tilde{r_2} = \frac{r_2 - a_2 \tilde{r_1}}{b_2 - a_2 \tilde{c_1}}, \quad \tilde{c_2} = \frac{c_2}{b_2 - a_2 \tilde{c_1}} \tag{31}$$

Entonces nos queda,

$$T_2 = \tilde{r_2} - \tilde{c_2} T_3 \tag{32}$$

En general, para $i = 2, 3, \ldots, n-1$ se tiene:

$$T_i = \tilde{r_i} - \tilde{c_i} T_{i+1}, \tag{33}$$

con:

$$\tilde{r_i} = \frac{r_i - a_i \tilde{r}_{i-1}}{b_i - a_i \tilde{c}_{i-1}}, \quad \tilde{c_i} = \frac{c_i}{b_i - a_i \tilde{c}_{i-1}}. \tag{34}$$

Para T_n se tiene:

$$T_n = \frac{r_n - a_n T_{n-1}}{b_n} \tag{35}$$

$$\Rightarrow T_n = \frac{r_n - a_n \left(\tilde{r}_{n-1} - \tilde{c}_{n-1} T_n\right)}{b_n} \tag{36}$$

$$\Rightarrow T_n \left(1 + \frac{a_n \tilde{c}_{n-1}}{b_n}\right) = \frac{r_n - a_n \tilde{r}_{n-1}}{b_n} \tag{37}$$

$$\Rightarrow T_n = \frac{r_n - a_n \tilde{r}_{n-1}}{b_n + a_n \tilde{c}_{n-1}} \tag{38}$$

$$\Rightarrow T_n = \tilde{r}_n \tag{39}$$

5. Metodologia computacional del caso 1D

Para el caso unidimensional se resuelve directamente el sistema tridiagonal producido por la discretización de Laplace:

$$A\mathbf{T} = \mathbf{r}. \tag{40}$$

Se implementa el algoritmo de Thomas para resolver este sistema de forma eficiente. Se usa el comando `gfortran tridiagonal.f90 calor1D.f90 -o calor1D` para compilar la versión secuencial. El programa se ejecuta con `./calor1D` y los resultados se guardan en archivos de texto para su posterior análisis.

Se muestra a continuación el código Fortran de la versión secuencial para el caso 1D. Se comienza abriendo el programa y declarando las variables necesarias para el algoritmo:

```
Program Calor1D
  Implicit none
  integer :: ii
  integer, parameter :: nn = 60
  integer, parameter :: L = 10 ! metros
  double precision :: deltax
  double precision, parameter :: q0_sobre_k = -1.d0
  double precision :: tt(nn)    ! vector de incognitas
  double precision :: rr(nn)    ! vector de resultados del s. ecuaciones
  double precision :: aa(nn), bb(nn), cc(nn) ! Variables para almacenar
  double precision :: cp(nn), den(nn)
```

Establecemos el dominio computacional y el paso de malla $\Delta x = L/nn$: El valor $L = 10$ metros define la longitud total del dominio de solución; se elige arbitrariamente para este problema de prueba. La discretización se realiza con $nn = 60$ nodos, resultando en $\Delta x = L/60$.

```
  ! Dominio computacional
  deltax  = dble(L)/nn
```

Luego, se inicializan los vectores necesarios para el algoritmo de Thomas. Se hace notar que no se construye la matriz completa A, sino que se almacenan solamente las diagonales a, b y c en los vectores correspondientes.

```
  ! Inicializacion de variables
  !
  aa(:)   = 0.d0
  bb(:)   = 0.d0
  cc(:)   = 0.d0
  rr(:)   = 0.d0
```

Se ensamblan los coeficientes de la matriz tridiagonal y el vector de resultados según la discretización de la ecuación de Laplace. Para los nodos interiores, los coeficientes son constantes: $a_i = 1$, $b_i = -2$ y $c_i = 1$.

```
  ! Ensamblar la matriz tridiagonal y el vector de resultados
  do ii = 2, nn-1
     !
```

```
      aa(ii) = 1.d0
      bb(ii) =-2.d0
      cc(ii) = 1.d0
      rr(ii) = 0.d0
      !
   end do
```

A continuación, se imponen las condiciones de frontera. En este caso, se asume una condición de Neumann en el extremo izquierdo ($x = 0$) con flujo q_0 y una condición de Dirichlet en el extremo derecho ($x = L$) con temperatura T_L. Esto se traduce en los coeficientes de la primera y última ecuaciones del sistema.

```
   ! Impone cond. frontera
   !
   aa(1)     = 0.d0 ! no se usa en los c'alculos
   bb(1)     =-1.d0
   cc(1)     = 1.d0
   rr(1)     = q0_sobre_k * deltax
   !
   aa(nn)    = 0.d0
   bb(nn)    = 1.d0
   cc(nn)    = 0.d0 ! no se usa en los c'alculos
   rr(nn)    = 0.d0
```

Se resuelve el sistema tridiagonal usando el algoritmo de Thomas, que se implementa en las subrutinas `tri_factor` y `tri_solve`. Estas subrutinas realizan la factorización y la resolución del sistema de forma eficiente, aprovechando la estructura tridiagonal de la matriz.

```
   ! Resolver el problema algebraico
   !
   call tri_factor(aa,bb,cc,cp,den,nn)
   call tri_solve(aa,cp,den,rr,tt,nn)
```

Finalmente, se guarda el resultado en un archivo de texto para su posterior análisis. Se escribe el vector de temperaturas tt junto con las posiciones correspondientes en el dominio.

```
   ! Postproceso
   !
   do ii = 1, nn
      write(101,*) (ii-1)*deltax, tt(ii)
   end do
   !
end Program Calor1D
```

Subrutinas: `tri, tri_solver y tri_factor:`

La subrutina `tri` resuelve el sistema tridiagonal completo en un solo paso: primero realiza la eliminación gaussiana hacia adelante y luego la sustitución hacia atrás.

Barrido hacia adelante (`gausselim`): Para $i = 2, 3, \ldots, n$, se calcula el multiplicador de eliminación y se actualizan la diagonal y el lado derecho *in-place*:

$$m_i = \frac{a_i}{b_{i-1}}, \tag{41}$$

$$b_i \leftarrow b_i - m_i\, c_{i-1}, \tag{42}$$

$$r_i \leftarrow r_i - m_i\, r_{i-1}. \tag{43}$$

Tras este barrido el sistema original $A\mathbf{u} = \mathbf{r}$ ha quedado reducido a uno triangular superior equivalente $U\mathbf{u} = \mathbf{r}^*$, donde $\mathbf{r}^*$ denota el lado derecho transformado.

Sustitución hacia atrás (`sustatras`): Se obtiene la última incógnita directamente y luego se retrocede:

$$u_n = \frac{r_n^*}{b_n^*}, \tag{44}$$

$$u_i = \frac{r_i^* - c_i\, u_{i+1}}{b_i^*}, \quad i = n-1,\, n-2,\, \ldots,\, 1. \tag{45}$$

El coste total es $\mathcal{O}(n)$ operaciones, frente a los $\mathcal{O}(n^3)$ de una eliminación gaussiana general.

Subrutina tri_factor:

Se explica a continuación el código de la subrutina `tri_factor`, que realiza la factorización de Thomas para una matriz tridiagonal constante.

Se declaran las variables de entrada y salida, incluyendo los vectores de coeficientes de la matriz tridiagonal y los vectores para almacenar la factorización.

```
  subroutine tri(a,b,c,r,u,n)
  !
  ! Subrutina que resuelve el problema
  !
  ! Ax=r donde A es una matriz tridiagonal con
  ! elementos a, b, c
  ! y devuelve el valor de u
  !
  implicit none
  !
  integer,          intent(in)    :: n
  double precision, intent(in)    :: a(n),c(n)
  double precision, intent(inout) :: b(n),r(n)
  double precision, intent(inout) :: u(n)
  integer :: i
  double precision :: m

  ! eliminacion elementos bajo la matriz
  gausselim: do i=2,n
     m = a(i)/b(i-1)
     r(i)=r(i)-m*r(i-1)
     b(i)=b(i)-m*c(i-1)
  end do gausselim
  !
  ! solucion para u
  !
  u(n)=r(n)/b(n)
  sustatras: do i=n-1,1,-1
     u(i)=(r(i)-c(i)*u(i+1))/b(i)
  end do sustatras
  !
end subroutine tri

subroutine tri_factor(a,b,c,cp,den,n)
  !
  ! Factorizacion de Thomas para una matriz tridiagonal constante.
  !
  implicit none
  !
  integer,          intent(in)  :: n
  double precision, intent(in)  :: a(n),b(n),c(n)
```

```
  double precision, intent(out) :: cp(n),den(n)
  integer :: i

  den(1) = b(1)
  cp(1)  = c(1)/den(1)

  do i=2,n-1
     den(i) = b(i) - a(i)*cp(i-1)
     cp(i) = c(i)/den(i)
  end do
  den(n) = b(n) - a(n)*cp(n-1)
  cp(n) = 0.d0
end subroutine tri_factor

subroutine tri_solve(a,cp,den,r,u,n)
  !
  ! Sustitucion usando la factorizacion calculada por tri_factor.
  ! Modifica r in-place para guardar el lado derecho transformado.
  !
  implicit none
  !
  integer,          intent(in)    :: n
  double precision, intent(in)    :: a(n),cp(n),den(n)
  double precision, intent(inout) :: r(n)
  double precision, intent(inout) :: u(n)
  integer :: i

  r(1) = r(1)/den(1)
  do i=2,n
     r(i) = (r(i) - a(i)*r(i-1))/den(i)
  end do

  u(n) = r(n)
  do i=n-1,1,-1
     u(i) = r(i) - cp(i)*u(i+1)
  end do
end subroutine tri_solve
```

6. Discretización del caso 2D

En el caso bidimensional, la ecuación de Laplace se discretiza como:

$$\frac{T_{i+1,j} - 2T_{i,j} + T_{i-1,j}}{(\Delta x)^2} + \frac{T_{i,j+1} - 2T_{i,j} + T_{i,j-1}}{(\Delta y)^2} = 0. \tag{46}$$

Separando los términos en la dirección x:

$$\frac{1}{(\Delta x)^2} T_{i-1,j} - 2\left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2}\right) T_{i,j} + \frac{1}{(\Delta x)^2} T_{i+1,j} = -\frac{T_{i,j-1} + T_{i,j+1}}{(\Delta y)^2}. \tag{47}$$

Esta expresión genera un sistema tridiagonal por cada linea horizontal j:

$$a_i^{(x)} T_{i-1,j} + b_i^{(x)} T_{i,j} + c_i^{(x)} T_{i+1,j} = r_i^{(x)}. \tag{48}$$

Los coeficientes usados son:

$$a_i^{(x)} = \frac{1}{(\Delta x)^2}, \tag{49}$$

$$b_i^{(x)} = -2\left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2}\right), \tag{50}$$

$$c_i^{(x)} = \frac{1}{(\Delta x)^2}, \tag{51}$$

$$r_i^{(x)} = -\frac{T_{i,j-1} + T_{i,j+1}}{(\Delta y)^2}. \tag{52}$$

De forma análoga, separando los términos en la dirección y:

$$\frac{1}{(\Delta y)^2}T_{i,j-1} - 2\left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2}\right)T_{i,j} + \frac{1}{(\Delta y)^2}T_{i,j+1} = -\frac{T_{i-1,j} + T_{i+1,j}}{(\Delta x)^2}. \tag{53}$$

Esto produce sistemas tridiagonales verticales:

$$a_j^{(y)}T_{i,j-1} + b_j^{(y)}T_{i,j} + c_j^{(y)}T_{i,j+1} = r_j^{(y)}, \tag{54}$$

con:

$$a_j^{(y)} = \frac{1}{(\Delta y)^2}, \tag{55}$$

$$b_j^{(y)} = -2\left(\frac{1}{(\Delta x)^2} + \frac{1}{(\Delta y)^2}\right), \tag{56}$$

$$c_j^{(y)} = \frac{1}{(\Delta y)^2}, \tag{57}$$

$$r_j^{(y)} = -\frac{T_{i-1,j} + T_{i+1,j}}{(\Delta x)^2}. \tag{58}$$

El proceso iterativo se detiene cuando el cambio entre dos iteraciones consecutivas es menor que una tolerancia prescrita. El residuo usado es:

$$R^{(k)} = \left[\sum_{i=1}^{n_x}\sum_{j=1}^{n_y}\left(T_{i,j}^{(k)} - T_{i,j}^{(k-1)}\right)^2\right]^{1/2}. \tag{59}$$

El criterio de convergencia es:

$$R^{(k)} < \varepsilon. \tag{60}$$

7. Metodologia computacional secuencial del caso 2D

El algoritmo secuencial para el caso 2D se implementa en el programa `secuencial_calor1D_calor2D.f90`. Se resuelve iterativamente el sistema de ecuaciones generado por la discretización bidimensional de la ecuación de Laplace. Se usa el mismo algoritmo de Thomas para resolver los sistemas tridiagonales en cada dirección. El programa se ejecuta con `./calor2D` y los resultados se guardan en archivos de texto para su posterior análisis.

La versión paralela distribuye los barridos independientes entre hilos de OpenMP. Si t_p es el tiempo de ejecución usando p hilos y t_1 es el tiempo de referencia secuencial, el speedup se calcula como:

$$S_p = \frac{t_1}{t_p}. \tag{61}$$

La eficiencia paralela mide qué fracción del uso ideal de los p hilos se aprovecha:

$$E_p = \frac{S_p}{p}. \tag{62}$$

El caso ideal corresponde a:

$$S_p^{ideal} = p, \quad E_p^{ideal} = 1. \tag{63}$$

8. Resultados

Las tablas de rendimiento se guardan en `docs/tables/` y las figuras en `docs/figures/`. Para cada número de hilos p, el tiempo reportado se puede obtener como el promedio de varias corridas:

$$\bar{t}_p = \frac{1}{N} \sum_{\ell=1}^{N} t_{p,\ell}, \tag{64}$$

donde N es el número de repeticiones y $t_{p,\ell}$ es el tiempo medido en la corrida ℓ usando p hilos. Con este promedio, el speedup experimental queda:

$$S_p^{real} = \frac{\bar{t}_1}{\bar{t}_p}. \tag{65}$$

La eficiencia experimental se calcula como:

$$E_p^{real} = \frac{S_p^{real}}{p} = \frac{\bar{t}_1}{p\,\bar{t}_p}. \tag{66}$$

Para comparar con la aceleración ideal se usa:

$$\Delta S_p = S_p^{ideal} - S_p^{real} = p - S_p^{real}. \tag{67}$$

Una diferencia ΔS_p cercana a cero indica que la ejecución paralela se aproxima al comportamiento ideal. Cuando ΔS_p crece, la eficiencia baja por costos de sincronización, acceso a memoria, regiones no paralelas o desbalance entre hilos.

8.1. Comparación final con el código de referencia

Se comparó la versión paralela desarrollada en `paralelo/src/` contra el código de referencia proporcionado en `paralelo/reference/`. Los archivos de referencia no fueron modificados. Para reducir el efecto de arranques fríos y procesos de fondo, se usó una corrida de calentamiento por número de hilos y se reportó la mediana de seis corridas medidas.

Para aislar el efecto del paralelismo bajo el mismo comportamiento numérico, en la versión desarrollada se reprodujeron dos errores presentes en `reference/`: (i) la condición de frontera inferior del barrido en dirección y se dejó con la misma formulación incorrecta, y (ii) el corte por convergencia basado en residuo se mantuvo desactivado, forzando la ejecución hasta `itermax`.

La comparación debe leerse como una comparación de comportamiento paralelo en la misma máquina, no como igualdad estricta de carga numérica: ambas versiones usan una malla de 480×240.

Cuadro 1: Comparación de tiempos y aceleración en la misma máquina.

Hilos	t_{nuestra} [s]	S_{nuestra}	t_{ref} [s]	S_{ref}
1	40.466	1.000	36.954	1.000
2	21.829	1.854	21.855	1.691
3	15.465	2.617	15.736	2.348
4	13.534	2.990	14.136	2.614
5	12.845	3.150	10.911	3.387
6	11.342	3.568	9.832	3.759
7	8.554	4.731	8.890	4.157
8	8.102	4.994	10.386	3.558

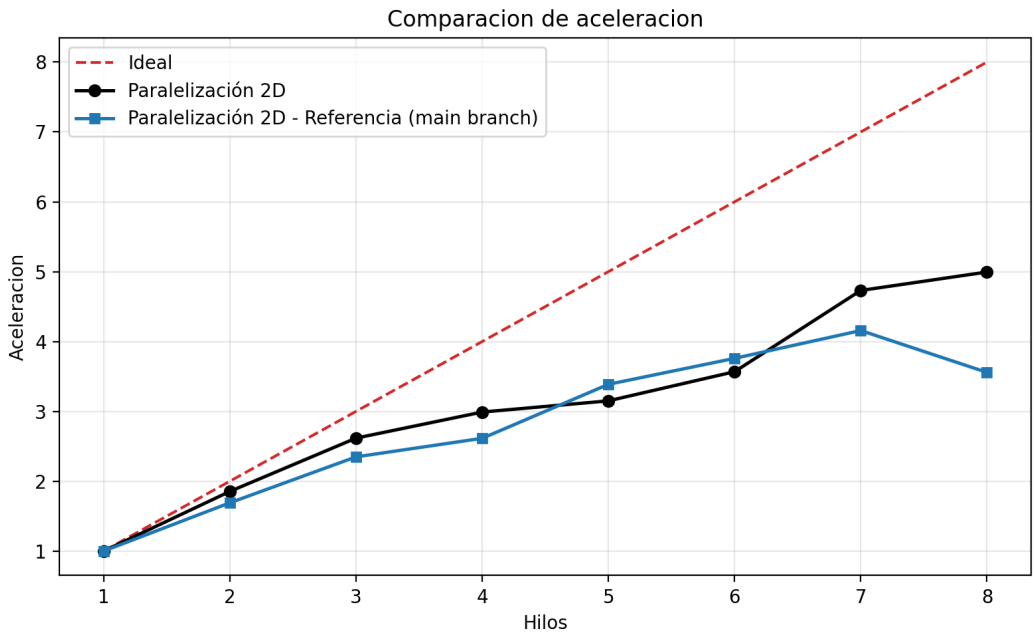

Figura 1: Curvas de aceleración de la versión desarrollada y la referencia.

9. Conclusiones